

Assignment 3

Due Thursday September 27 at 11:59pm on Quercus

As before, the questions without solutions (here, the last two) are an assignment: you need to do these questions yourself and hand them in (instructions below).

The assignment is due on the date shown above. An assignment handed in after the deadline is late, and may or may not be accepted (see course outline). My solutions to the assignment questions will be available when everyone has handed in their assignment.

You are reminded that work handed in with your name on it must be *entirely your own work*.

Assignments are to be handed in on Quercus. See <https://www.utsc.utoronto.ca/~butler/c32/quercus1.nb.html> for instructions on handing in assignments in Quercus. Markers' comments and grades will be available there as well.

As ever, you'll want to begin with:

```
library(tidyverse)
```

1. Do children (aged 8–17) spend more time on electronic devices now than they did 10 years ago? Samples of 15 children aged 8–17 were taken in each of two years, 1999 and 2009, and the children (with their parents' help) were asked to keep a diary of the number of hours they spent using electronic devices on a certain day. The data are in the file <http://www.utsc.utoronto.ca/~butler/c32/pluggedin.txt>.
 - (a) Read in the data and verify that you have 30 rows of data from two different years.

Solution: I see this:

```

myurl="http://www.utoronto.ca/~butler/c32/pluggedin.txt"
plugged=read_delim(myurl, " ")

## Parsed with column specification:
## cols(
##   year = col_integer(),
##   hours = col_integer()
## )

plugged

## # A tibble: 30 x 2
##   year hours
##   <int> <int>
## 1  1999     4
## 2  1999     5
## 3  1999     7
## 4  1999     7
## 5  1999     5
## 6  1999     7
## 7  1999     5
## 8  1999     6
## 9  1999     5
##10  1999     6
## # ... with 20 more rows

```

I see only the first ten rows (with an indication that there are 20 more, so 30 altogether). In your notebook, it'll look a bit different: again, you'll see the first 10 rows, but you'll see exactly how many rows and columns there are, and there will be buttons "Next" and "Previous" to see earlier and later rows, and a little right-arrow to see more columns to the right (to which is added a little left-arrow if there are previous columns to scroll back to). If you want to check for yourself that there are 30 rows, you can click Next a couple of times to get down to row 30, and then see that the Next button cannot be clicked again, and therefore that 30 rows is how many there are.

Or, you can summarize the years by counting how many there are of each:

```

plugged %>% count(year)

## # A tibble: 2 x 2
##   year     n
##   <int> <int>
## 1  1999    15
## 2  2009    15

```

or the more verbose form of the same thing:

```

plugged %>% group_by(year) %>% summarize(rows=n())

## # A tibble: 2 x 2
##   year rows
##   <int> <int>
## 1  1999    15
## 2  2009    15

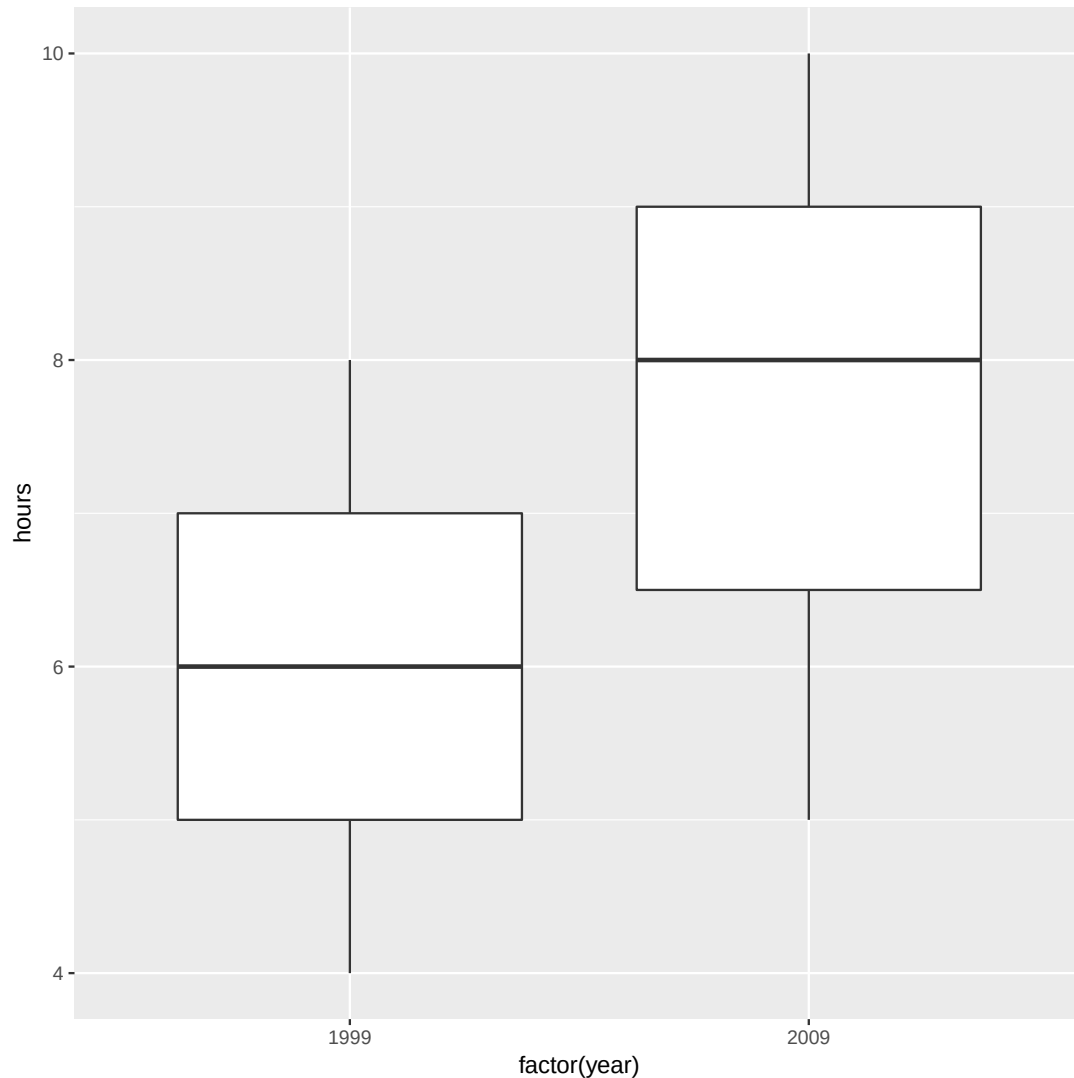
```

Any of those says that it looks good. 30 rows, 1999 and 2009, 15 measurements for each.

- (b) Draw side-by-side boxplots of the number of hours for each year. `year` is a numeric variable that we want to treat as a factor, so we need to *make* it into a factor.

Solution:

```
ggplot(plugged,aes(x=factor(year),y=hours))+geom_boxplot()
```



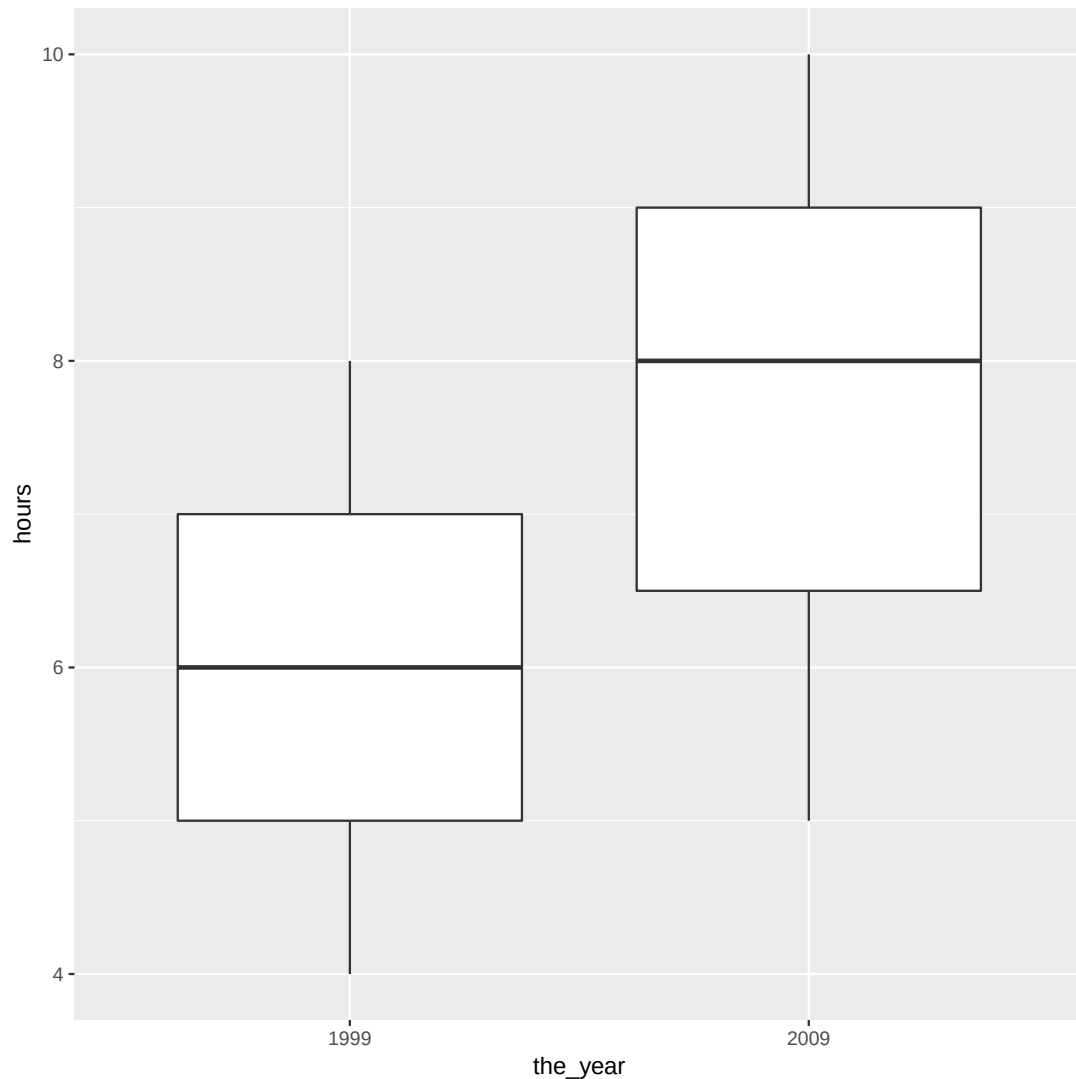
The `fct_inorder` trick from assignment 1 will also work, since the years are in the data in the order we want them to be displayed.

The median for 2009 is noticeably higher, and there is no skewness or outliers worth worrying about.

The measurements for the two years have a very similar spread, so there would be no problem running the pooled test here.

You might be bothered by the `factor(year)` on the *x*-axis. To get around that, you can define year-as-factor *first*, using `mutate`, then feed your new column into the boxplot. That goes like this. There is a wrinkle that I explain afterwards:

```
plugged %>% mutate(the_year=factor(year)) %>%
  ggplot(aes(x=the_year, y=hours))+geom_boxplot()
```

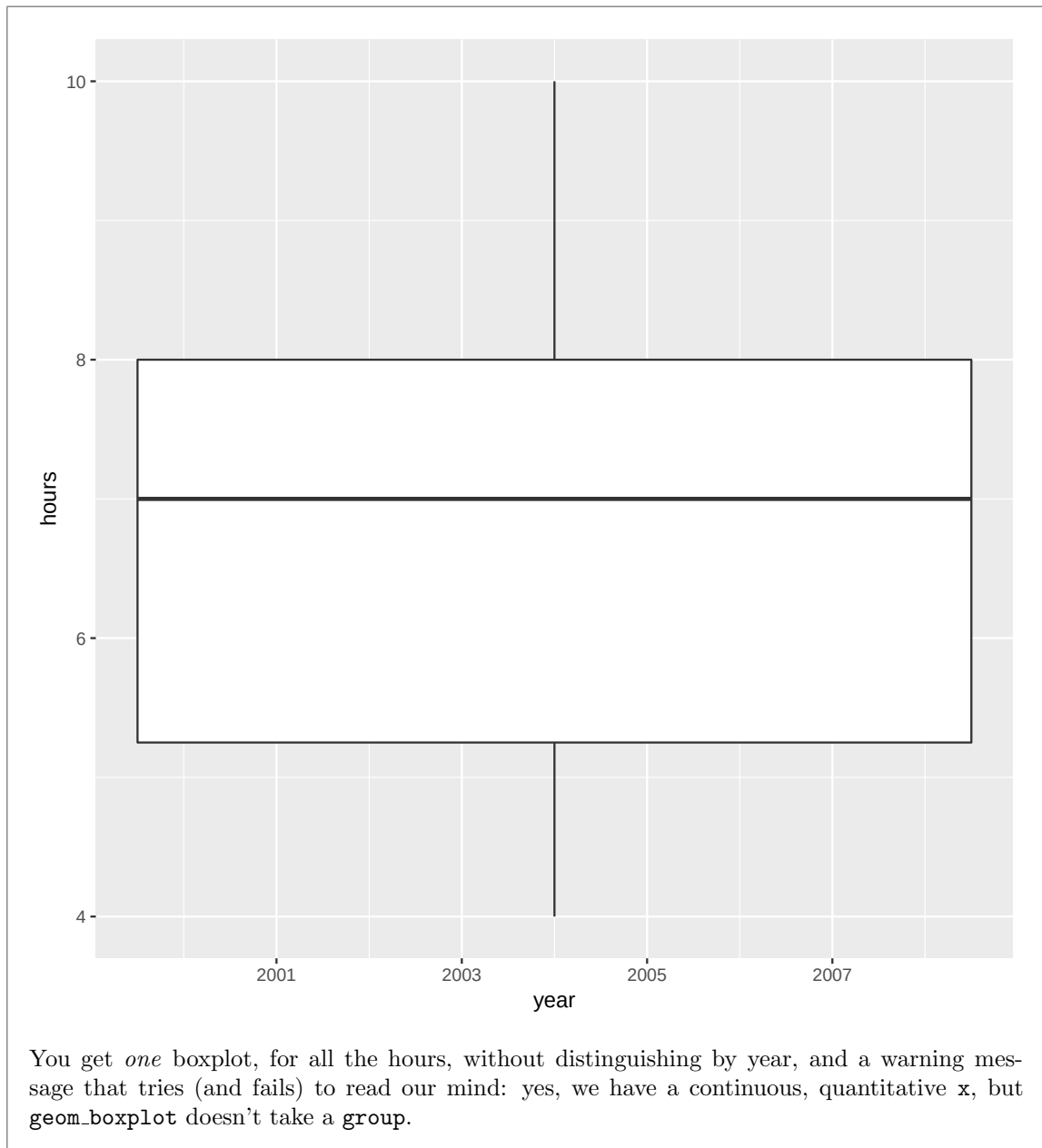


You could even redefine `year` to be the factor version of itself (if you don't need the year-as-number anywhere else). The wrinkle I mentioned above is that in the `ggplot` you *do not* name the data frame first; the data frame used is the (nameless) data frame that came out of the previous step, not `plugged` but `plugged` with a new column `the_year`.

Note how the *x*-axis now has the name of the new variable.

If you forget to make `year` into a factor, this happens:

```
ggplot(plugged,aes(x=year,y=hours))+geom_boxplot()
## Warning: Continuous x aesthetic -- did you forget aes(group=...)?
```



(c) Test whether the mean number of hours has *increased* since 1999. Which test did R do?

Solution: The hard part to remember is how you specify a one-sided test in R; it's `alternative="less"` (rather than "greater") because 1999 is "before" 2009:

```
t.test(hours~year,data=plugged,alternative="less")

##
##  Welch Two Sample t-test
##
## data:  hours by year
## t = -3.3323, df = 24.861, p-value = 0.001348
## alternative hypothesis: true difference in means is less than 0
## 95 percent confidence interval:
##      -Inf -0.8121415
## sample estimates:
## mean in group 1999 mean in group 2009
##      5.933333      7.600000
```

The P-value is 0.0013. R does the Welch-Satterthwaite test by default (the unequal-variances one). Since we didn't change that, that's what we got. (The pooled test is below.)

This is the cleanest way to do it, because this version of `t.test`, with a “model formula” (the thing with the squiggle) allows a `data=` to say which data frame to get things from. The other ways, using (for example) `with`, also work:

```
with(plugged,t.test(hours~year,alternative="less"))

##
##  Welch Two Sample t-test
##
## data:  hours by year
## t = -3.3323, df = 24.861, p-value = 0.001348
## alternative hypothesis: true difference in means is less than 0
## 95 percent confidence interval:
##      -Inf -0.8121415
## sample estimates:
## mean in group 1999 mean in group 2009
##      5.933333      7.600000
```

This also works, but is *ugly*:

```
t.test(plugged$hours~plugged$year,alternative="less")

##
##  Welch Two Sample t-test
##
## data:  plugged$hours by plugged$year
## t = -3.3323, df = 24.861, p-value = 0.001348
## alternative hypothesis: true difference in means is less than 0
## 95 percent confidence interval:
##      -Inf -0.8121415
## sample estimates:
## mean in group 1999 mean in group 2009
##      5.933333      7.600000
```

Ugly because you've just typed the name of the data frame and the dollar sign *twice* for no reason. As a general principle, if you as a programmer are repeating yourself, you should stop and ask yourself how you can avoid the repeat.

If you want the pooled test in R, you have to ask for it:

```
t.test(hours~year,alternative="less",data=plugged,var.equal=T)

##
## Two Sample t-test
##
## data:  hours by year
## t = -3.3323, df = 28, p-value = 0.001216
## alternative hypothesis: true difference in means is less than 0
## 95 percent confidence interval:
##      -Inf -0.8158312
## sample estimates:
## mean in group 1999 mean in group 2009
##      5.933333      7.600000
```

As is often the case, the P-values for the pooled and Welch-Satterthwaite tests are very similar, so from that point of view it doesn't matter much which one you use. If you remember back to the boxplots, the number of hours had about the same spread for the two years, so if you used the pooled test instead of the Welch-Satterthwaite test, that would have been just fine.

There is a school of thought that says we should learn the Welch-Satterthwaite test and use that always. This is because W-S (a) works when the populations from which the groups are sampled have different SDs and (b) is pretty good even when those SDs are the same.

The pooled test can go badly wrong if the groups have very different SDs. The story is this: if the larger sample is from the population with the larger SD, the probability of a type I error will be smaller than α , and if the larger sample is from the population with the *smaller* SD, the probability of a type I error will be larger than α . This is why you see S-W in STAB22. You see the pooled test in STAB57 because the logic of its derivation is so much clearer,¹ not because it's really the better test in practice. The theory says that if your data are normal in both groups with the same variance, then the pooled test is best, but it says *nothing* about the quality of the pooled test if any of that goes wrong.

The usual approach to assessing things like this is via simulation, as we do for estimating power (later): generate some random data eg. from normal distributions with the same means, SDs 10 and 20 and sample sizes 15 and 30, run the pooled *t*-test, see if you reject, then repeat lots of times and see whether you reject about 5% of the time. Then do the same thing again with the sample sizes switched around. Or, do the same thing with Welch-Satterthwaite.

- (d) Obtain a 99% confidence interval for the difference in means.

Solution: Take off the thing that made it one-sided, and put in a thing that gets the right CI:

```
t.test(hours~year,data=plugged,conf.level=0.99)

##
## Welch Two Sample t-test
##
## data:  hours by year
## t = -3.3323, df = 24.861, p-value = 0.002696
## alternative hypothesis: true difference in means is not equal to 0
## 99 percent confidence interval:
##  -3.0614628 -0.2718705
## sample estimates:
## mean in group 1999 mean in group 2009
##           5.933333           7.600000
```

−3.06 to −0.27. The interval contains only negative values, which is consistent with our having rejected a null hypothesis of no difference in means.

2. In 2009, the Toronto Star commissioned a survey to address the issue of who is better at parking a car: men or women. The researchers recorded 93 drivers who were parallel-parking their car in downtown Toronto, and for each driver, recorded the distance between the car and the curb, in inches, when the driver was finished parking their car. The data are in an Excel spreadsheet, <http://www.utsc.utoronto.ca/~butler/c32/parking.xlsx>. (Let me know if you cannot handle this format.) Click on the link. The data will probably download automatically. Check the folder on your computer where things get downloaded.² If the spreadsheet is just displayed and not downloaded, save it somewhere on your computer.
 - (a) There are two sheets in this spreadsheet workbook. They are of the same data in two different formats. Take a look at Sheet 1 and Sheet 2. Describe the format of the data in each case. Which will be the most suitable data layout, bearing in mind that one of the first things we do is to make side-by-side boxplots of parking distances for males and females? Explain briefly.

Solution: The data in Sheet 1 has one column of parking distances for males, and another for females. This is often how you see data of this sort laid out. Sheet 2 has *one* column of parking distances, all combined together, and a second column indicating the gender of the driver whose distance is in the first column.

If you look back at the kind of data we’ve used to make side-by-side boxplots, it’s always been in the format of Sheet 2: one column containing all the values of the variable we’re interested in, with a second column indicating which group each observation belongs to (“group” here being “gender of driver”). So we need to use the data in Sheet 2, because the data in Sheet 1 are not easy to handle with R.

The layout of Sheet 2 is the way R likes to do most things: so-called “long format” with a lot of rows and not many columns. This is true for descriptive stuff: side-by-side boxplots or histograms or means by group, as well as modelling such as (here) a two-sample *t*-test, or (in other circumstances, as with several groups) a one-way analysis of variance.

Hadley Wickham, the guy behind the **tidyverse**, likes to talk about “tidy data” (like Sheet 2), with each column containing a variable, and “untidy data” (like Sheet 1), where the two columns are the same thing (distances), but under different circumstances (genders). As we’ll see later, it is possible to convert from one format to the other. Usually you want to make untidy data tidy (the function for this is called **gather**).

- (b) Read your preferred sheet directly into R, *without* using a `.csv` file. (There is a clue in the lecture notes, in the section about reading in files.) If you get stuck, make a `.csv` file and read that in.

Solution: The direct way is to use the package `readxl`. This has a `read_excel` that works the same way as any of the other `read_` functions. You'll have to make sure that you read in sheet 2, since that's the one you want. If the first line below gives you an error, you probably want this first:

```
install.packages("readxl")
```

and then go ahead:

```
library(readxl)
parking=read_excel("parking.xlsx",sheet=2)
parking

## # A tibble: 93 x 2
##   distance gender
##   <dbl> <chr>
## 1     0.5 male
## 2     1   male
## 3     1.5 male
## 4     1.5 male
## 5     1.5 male
## 6     3   male
## 7     3.5 male
## 8     5   male
## 9     6   male
## 10    6   male
## # ... with 83 more rows
```

You have to do it this way, using the version of the spreadsheet on your computer, since `read_excel` won't take a URL, or if it does, I can't make it work.³ I put the spreadsheet in R Studio's current folder, so I could read it in by name, or you can do the `f=file.choose()` thing, find it, then read it in.

The `sheet=` thing can take either a number (as here: the second sheet in the workbook), or a name (whatever name the sheet has on its tab in the workbook).

If you can't make this work, then do it in two steps: save the appropriate sheet as a `.csv` file, and then read the `.csv` file using `read_csv`. If you experiment, you'll find that saving a spreadsheet workbook as `.csv` only saves the sheet you're looking at, so make sure you are looking at sheet 2 before you Save As `.csv`. I did that, and called my saved `.csv` `parking2.csv` (because it was from sheet 2, but you can use any name you like).

Then I read this into R thus:

```

parking2=read_csv("parking2.csv")

## Parsed with column specification:
## cols(
##   distance = col_double(),
##   gender = col_character()
## )

parking2

## # A tibble: 93 x 2
##   distance gender
##   <dbl> <chr>
## 1     0.5 male
## 2     1   male
## 3     1.5 male
## 4     1.5 male
## 5     1.5 male
## 6     3   male
## 7     3.5 male
## 8     5   male
## 9     6   male
## 10    6   male
## # ... with 83 more rows

```

The read-in data frame `parking` has 93 rows ($47 + 46 = 93$ drivers) and two columns: the distance from the curb that the driver ended up at, and the gender of the driver. This is as the spreadsheet Sheet 2 was, and the first few distances match the ones in the spreadsheet.

If I were grading this, you'd get some credit for the `.csv` route, but I really wanted you to figure out how to read the Excel spreadsheet directly, so that's what would be worth full marks.

You might want to check that you have some males and some females, and how many of each, which you could do this way:

```

parking %>% count(gender)

## # A tibble: 2 x 2
##   gender     n
##   <chr> <int>
## 1 female   47
## 2 male    46

```

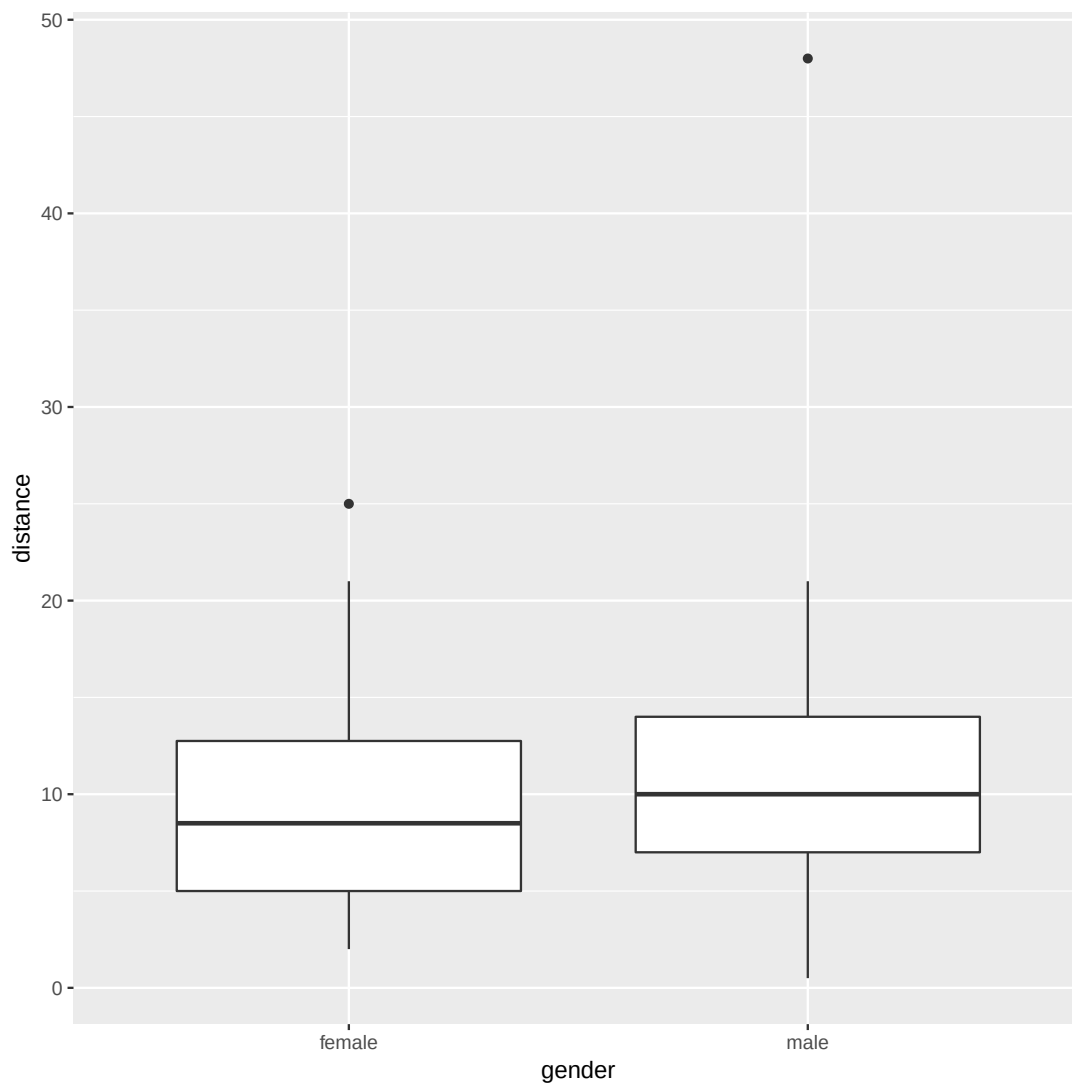
- (c) Obtain side-by-side boxplots of parking distances for males and females. Does one gender seem to be better at parking than the other? Explain briefly.

Solution: With the right data set, this is a piece of cake:

```

ggplot(parking, aes(x=gender, y=distance))+geom_boxplot()

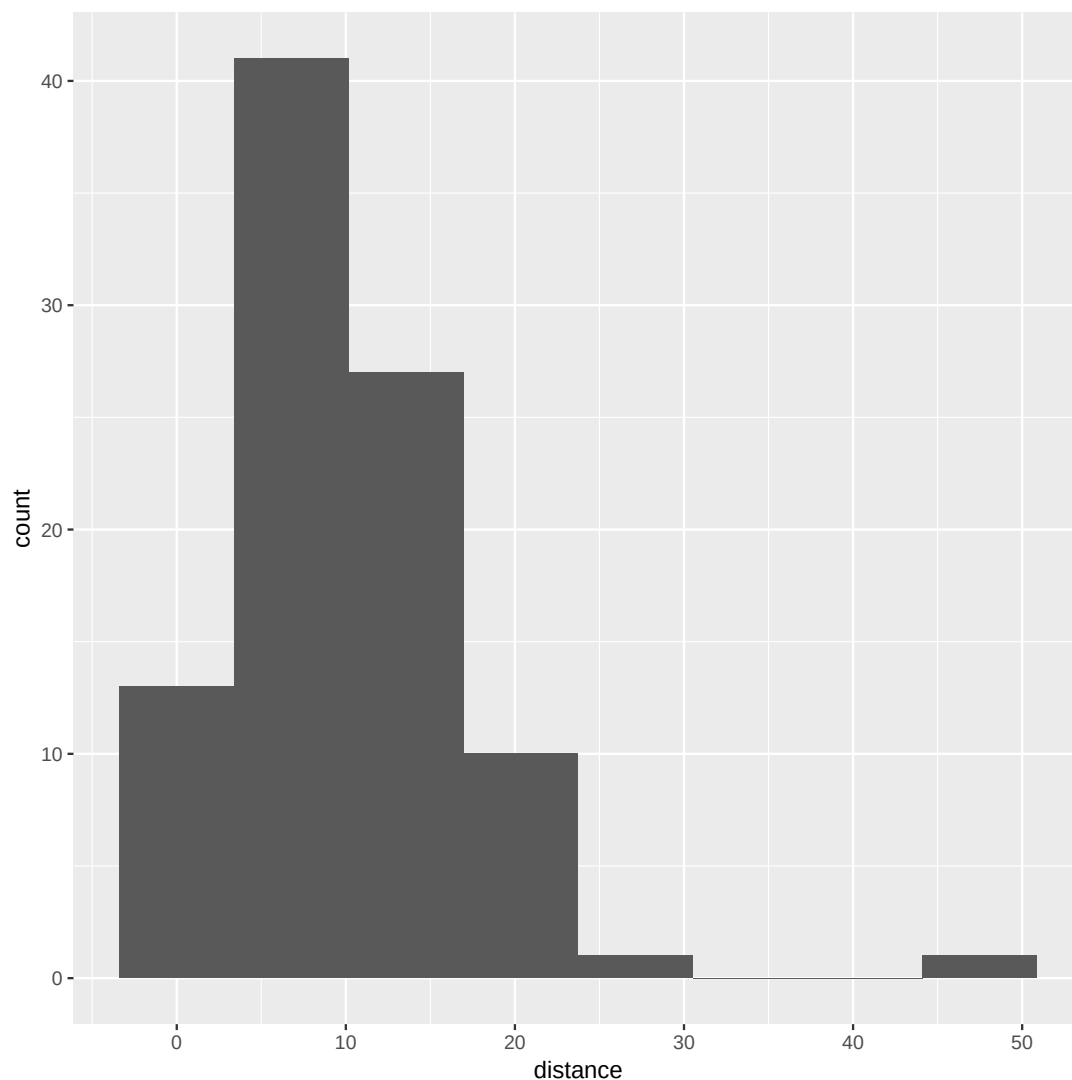
```



The outcome variable is distance from the curb, so smaller should be better (more accurate parking). With that in mind, the median for females is a little smaller than for males (about 8.5 vs. about 10), so it seems that on average females are more accurate parkers than males are. The difference is small, however (and so you might be wondering at this point whether it's a statistically significant difference — don't worry, that's coming up).

Before I leave this one, I want to show you something else: above-and-below histograms, as another way of comparing males and females (two or more groups, in general). First, we make a histogram of all the distances, without distinguishing by gender:

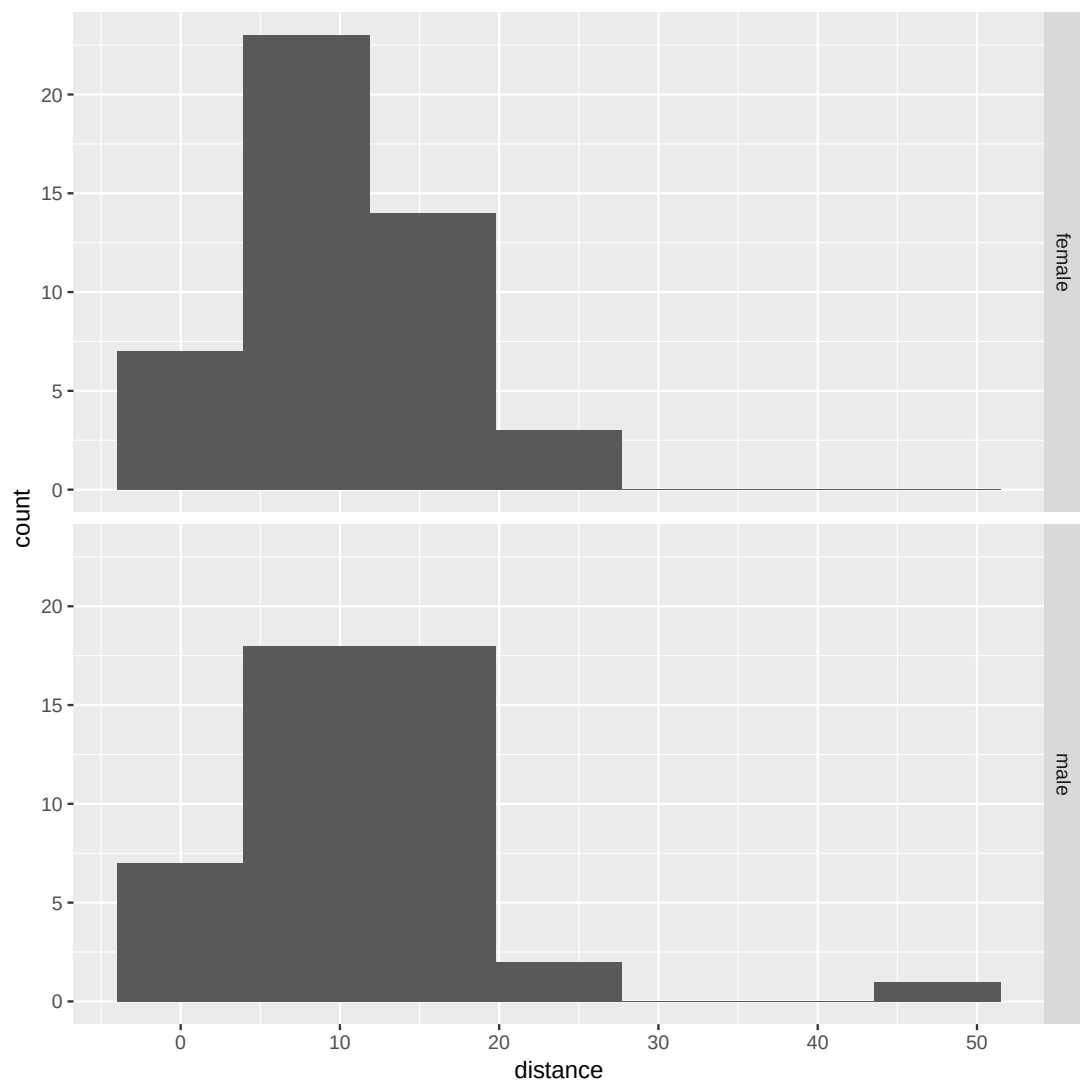
```
ggplot(parking, aes(x=distance))+geom_histogram(bins=8)
```



That big outlier is the very inaccurate male driver.

Now, how do we get a *separate* histogram for each **gender**? In **ggplot**, separate plots for each “something” are called **facets**, and the way to get facets arranged as you want them is called **facet_grid**. Let me show you the code first, and then explain how it works:

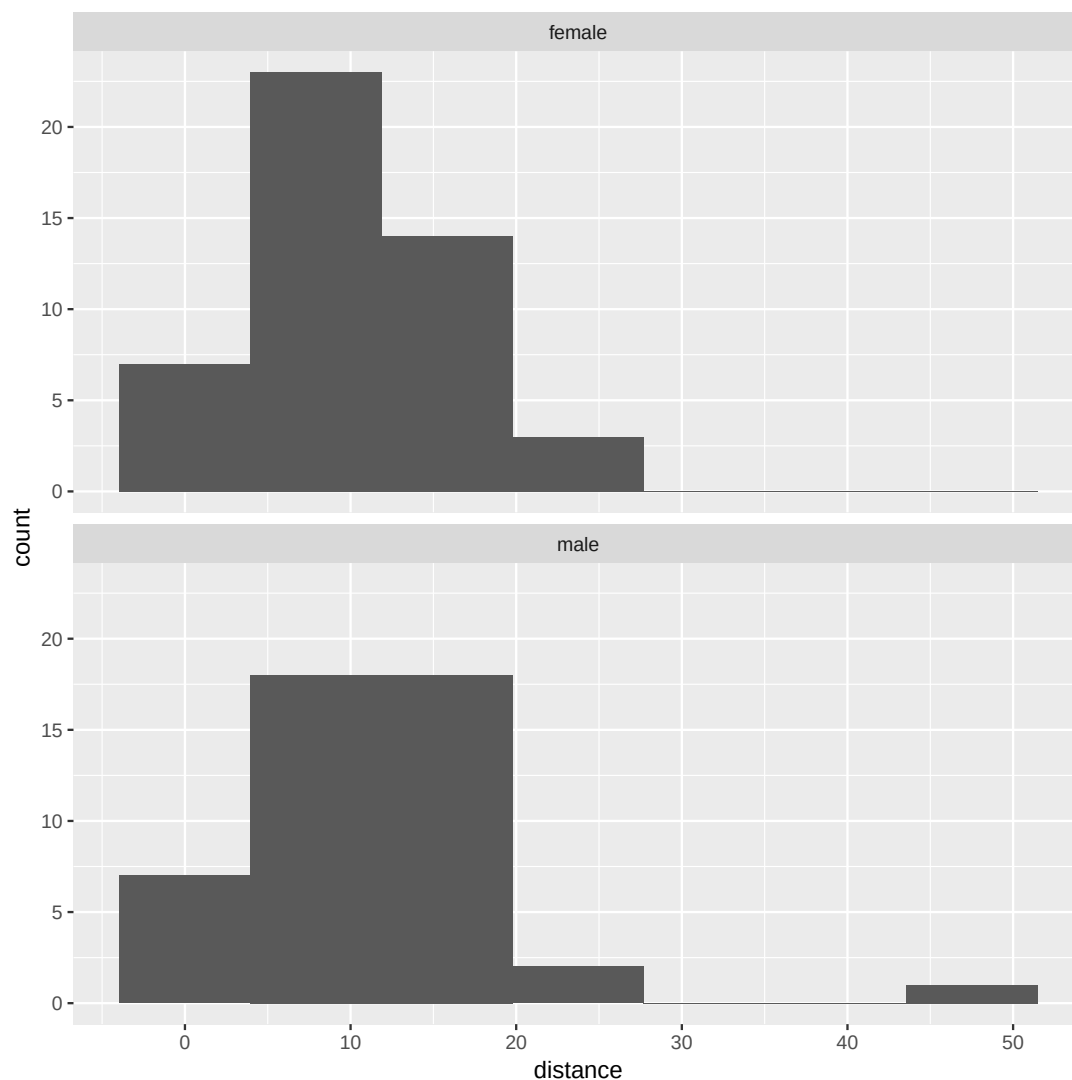
```
ggplot(parking, aes(x=distance)) +  
  geom_histogram(bins=7) +  
  facet_grid(gender~.)
```



`facet_grid` takes a “model formula” with a squiggle, with y on the left and x on the right. We want to compare our two histograms, one for males and one for females, and I think the best way to compare histograms is to have one on top of the other. Note that the same `distance` scale is used for both histograms, so that it is a fair comparison. The above-and-below is accomplished by having `gender` as the y in the arrangement of the facets, so it goes before the squiggle. We don’t have any x in the arrangement of the facets, and we tell `ggplot` this by putting a dot where the x would be.⁴

You can also use `facet_wrap` for this, but you have to be more careful since you don’t have any control over how the histograms come out (you probably get them side by side, which is not so helpful for comparing distributions). You can make it work by using `ncol=1` to arrange “all” the histograms in one column:

```
ggplot(parking, aes(x=distance)) +
  geom_histogram(bins=7) +
  facet_wrap(~gender, ncol=1)
```



The centres of both histograms are somewhere around 10, so it's hard to see any real difference between males and females here. Maybe this is further evidence that the small difference we saw between the boxplots is really not worth getting excited about.

You might be concerned about how you know what to put with the squiggle-thing in `facet_grid` and `facet_wrap`. The answer is that `facet_wrap` *only* has something to the right of the squiggle (which `ggplot` then decides how to arrange), but `facet_grid` *must* have something on *both* sides of the squiggle (how to arrange in the *y* direction on the left, how to arrange in the *x* direction on the right), and if you don't have anything else to put there, you put a dot. Here's my `facet_grid` code from above, again:

```
ggplot(parking, aes(x=distance))+  
  geom_histogram(bins=7)+  
  facet_grid(gender~.)
```

We wanted gender to go up and down, and we had nothing to go left and right, hence the dot. Contrast that with my `facet_wrap` code:⁵

```
ggplot(parking, aes(x=distance)) +  
  geom_histogram(bins=7) +  
  facet_wrap(~gender)
```

This says “make a separate facet for each gender”, but it doesn’t say anything about how to arrange them.

The choice of bins for my histogram(s) came from Sturges’ rule: with n being the number of observations, you use k bins where $k = \log_2(n) + 1$, rounded up. If we were to make a histogram of all the parking distances combined together, we would have $n = 47 + 48 = 95$ observations, so we should use this many bins:

```
sturges=log(95,2)+1  
sturges  
## [1] 7.569856
```

Round this up to 8. (The second thing in `log` is the base of the logs, if you specify it, otherwise it defaults to e and gives you “natural” logs.) I seem to have the powers of 2 in my head, so I can do it mentally by saying “the next power of 2 is 128, which is 2^7 , so I need $7 + 1 = 8$ bins.”

Or:

```
with(parking, nclass.Sturges(distance))  
## [1] 8
```

Sturges’ rule tends to produce not enough bins if n is small, so be prepared to increase it a bit if you don’t have much data.

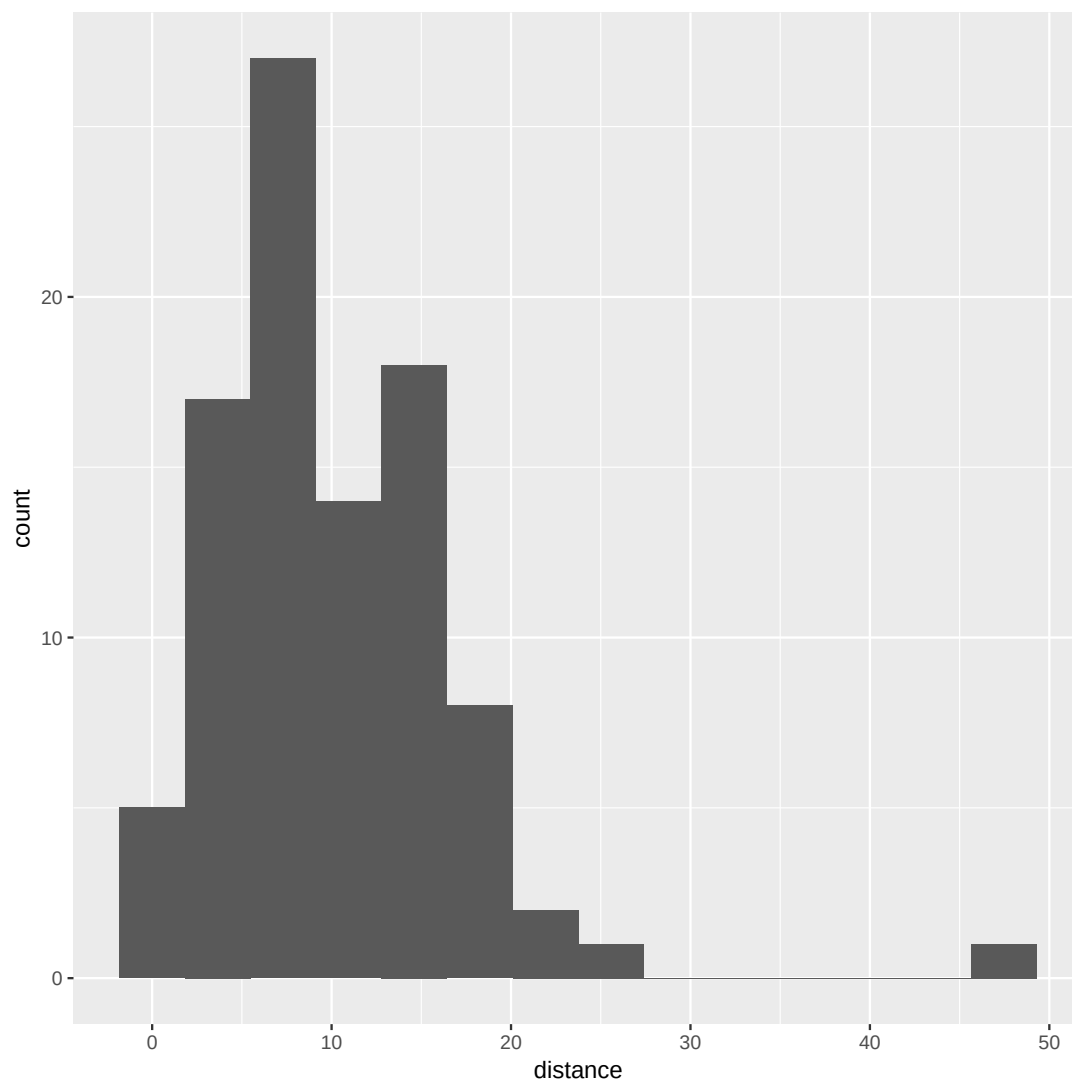
I think that gives a fairly bare-bones picture of the shape: skewed to the right with outlier.

The other rule we saw was Freedman-Diaconis:

```
with(parking, nclass.FD(distance))  
## [1] 14
```

and that leads to this histogram:

```
ggplot(parking, aes(x=distance)) + geom_histogram(bins=14)
```



That gives rather more detail (a lot more bars: the binwidth in the Sturges-rule histogram is about 7, or twice what you see here), but in this case the overall story is about the same.

In the case of faceted histograms, you would want to apply a rule that uses the number of observations in *each* histogram. The facets might have quite different numbers of observations, but you can only use one `binwidth` (or `bins`), so you may have to compromise. For example, using Sturges' rule based on 47 observations (the number of males; the number of females is one more):

```
log(47, 2)+1
## [1] 6.554589
```

and so each facet should have that many bins, rounded up. That's where I got my 7 for the faceted histogram from. This one doesn't work immediately with `nclass.Sturges`, because we do not have *one* column whose length is the number of observations we want: we have one column of distances that are males and females mixed) up. To do *that*, *filter* one of the genders first:


```
parking %>%  
  filter(gender=="female") %>%  
  with(., nclass.Sturges(distance))  
  
## [1] 7
```

I used the “dot” trick again, to say “use the data frame I just created”.

- (d) Explain briefly why this is two independent samples rather than matched pairs.

Solution:

There is no way to pair any male with a corresponding female, because they are unrelated people. You might also notice that there are not even the *same number* of males and females, so there can be no way of pairing them up without leaving one over. (In general, if the two samples are paired, there must be the same number of observations in each; if there are different numbers in each, as here, they cannot be paired.)

If you want that more mathematically, let n_1 and n_2 be the two sample sizes; then:

$$\text{Paired} \implies n_1 = n_2$$

from which it follows logically (the “contrapositive”) that

$$n_1 \neq n_2 \implies \text{not paired}$$

You’ll note from the logic that if the two sample sizes are the same, that tells you *nothing* about whether it’s paired or independent samples: it could be either, and in that case you have to look at the description of the data to decide between them.

Here, anything that gets at why the males and females cannot be paired up is good.

- (e) Run a suitable t -test for comparing parking distances for males and females. What do you conclude, in the context of the data?

Solution: A two-sample t -test. I think either the Welch or the pooled one can be justified (and I would expect them to give similar answers). You can do the Welch one either without comment or by asserting that the boxplots show different spreads; if you are going to do the pooled one, you need to say that the spreads are “about equal”, by comparing the heights of the boxes on the boxplots:

```
t.test(distance~gender,data=parking)

##
##  Welch Two Sample t-test
##
## data:  distance by gender
## t = -1.3238, df = 79.446, p-value = 0.1894
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -4.5884103  0.9228228
## sample estimates:
## mean in group female    mean in group male
##           9.308511           11.141304
```

This is the Welch-Satterthwaite version of the test, the one that does not assume equal SDs in the two groups. The P-value of 0.1894 is not small, so there is no evidence of any difference in parking accuracy between males and females.

Or, this being the pooled one:

```
t.test(distance~gender,data=parking,var.equal=T)

##
##  Two Sample t-test
##
## data:  distance by gender
## t = -1.329, df = 91, p-value = 0.1872
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -4.5722381  0.9066506
## sample estimates:
## mean in group female    mean in group male
##           9.308511           11.141304
```

You might have thought, looking at the boxplots, that the groups had about the same SD (based, for example, on noting that the two boxes were about the same height, so the IQRs were about the same). In that case, you might run a pooled t -test, which here gives an almost identical P-value of 0.1872, and the exact same conclusion.

(f) Why might you have some doubts about the t -test that you just did? Explain briefly.

Solution: The two-sample t -test is based on an assumption of normally-distributed data within each group. If you go back and look at the boxplots, you'll see either (depending on your point of view) that both groups are right-skewed, or that both groups have outliers, neither of which fits a normal distribution. The outlier in the male group is particularly egregious.⁶ So I think we are entitled to question whether a two-sample t -test is the right thing to do.

Having said that, we should go back and remember that the t -tests are “robust to departures from normality” (since we are working with the Central Limit Theorem here), and therefore that this test might be quite good even though the data are not normal, because the sample sizes of 40-plus are large (by the standards of what typically makes the Central Limit Theorem work for us). So it may not be as bad as it seems.

A common competitor for the two-sample t -test is the Mann-Whitney test. This doesn't assume

normality, but it *does* assume symmetric distributions, which it's not clear that we have here. I like a test called Mood's Median Test, which is kind of the two-sample equivalent of the sign test (which we will also see later). It goes like this:

Work out the overall median of all the distances, regardless of gender:

```
parking %>% summarize(med=median(distance))  
  
## # A tibble: 1 x 1  
##   med  
##   <dbl>  
## 1     9
```

The overall median is 9.

Count up how many distances of each gender were above or below the overall median. (Strictly, I'm supposed to throw away any values that are exactly equal to the overall median, but I won't here for clarity of exposition.)

```
tab=with(parking, table(gender, distance<9))  
tab  
  
##  
## gender    FALSE TRUE  
## female     23    24  
## male       27    19
```

For example, 19 of the male drivers had a distance (strictly) less than 9. Both genders are pretty close to 50-50 above and below the overall median, which suggests that the males and females have about the same median. This can be tested (it's a chi-squared test for independence, if you know that):

```
chisq.test(tab, correct=F)  
  
##  
## Pearson's Chi-squared test  
##  
## data:  tab  
## X-squared = 0.89075, df = 1, p-value = 0.3453
```

This is even less significant (P-value 0.3453) than the two-sample t -test, and so is consistent with our conclusion from before that there is actually no difference between males and females in terms of average parking distance. The Mood's median test is believable because it is not affected by outliers or distribution shape.

- (g) The Toronto Star in its report said that females are more accurate at parking their cars. Why do you think they concluded that, and do you think they were right to do so? Explain briefly.

Solution: The conclusion from the boxplots was that the female median distance was less than the males, slightly, in this sample. That is probably what the Star seized on.

Were they right? Well, that was why we did the test of significance. We were trying to see whether this observed difference between males and females was “real” (would hold up if you looked at “all” male and female drivers) or “reproducible” (you would expect to see it again if you did another study like this one). The large, non-significant P-values in all our tests tell us that the difference observed here was nothing more than chance. So it was not reasonable to

conclude that females generally are more accurate at parallel-parking than males are.

3. In opinion surveys (and other places), we are testing for a proportion p (for example, the proportion of people agreeing with some statement). Often, we want to know whether the proportion is “really” greater than 0.5.⁷ That would entail testing a null $H_0 : p = 0.5$ against an alternative $H_a : p > 0.5$. This is usually done by calculating the test statistic

$$z = \frac{\hat{p} - 0.5}{\sqrt{0.25/n}},$$

where $\hat{p}$ is the observed proportion in the sample, and getting a P-value from the upper tail of a standard normal distribution. (The 0.25 is $p(1-p)$ where $p = 0.5$.) This is what `prop.test` does, as we investigate shortly.

- (a) Use `rbinom` to generate a random value from a binomial distribution with $n = 100$ and $p = 0.6$. There are three inputs to `rbinom`: the first one should be the number 1, and the second and third are the n and p of the binomial distribution.

Solution: I am doing some preparatory work that you don’t need to do:

```
set.seed(457299)
```

By setting the “seed” for the random number generator, I guarantee that I will get the same answers every time I run my code below (and therefore I can talk about my answers without worrying that they will change). Up to you whether you do this. You can “seed” the random number generator with any number you like. A lot of people use 1. Mahinda seems to like 123. Mine is an old phone number.

And so to work:

```
rbinom(1,100,0.6)
## [1] 60
```

I got exactly 60% successes this time. You probably won’t get exactly 60, but you should get somewhere close. (If you use my random number seed and use the random number generator exactly the same way I did, you should get the same values I did.)

For fun, you can see what happens if you change the 1:

```
rbinom(3,100,0.6)
## [1] 58 57 55
```

Three random binomials, that happened to come out just below 60. We’re going to leave the first input as 1, though, and let `rerun` handle “lots of sampled values” later.

- (b) Using the random binomial that you generated just above, use `prop.test` to test whether it could reasonably have come from a binomial population with $n = 100$ and $p = 0.5$, or whether p is actually bigger than 0.5. (Of course, you know it actually did not come from a population with $p = 0.5$.)

`prop.test` has, for us, four inputs, thus:

- the observed number of successes
- the `n` of the binomial distribution
- the null-hypothesis `p` of the binomial distribution

- the alternative hypothesis, here “greater”

Solution: I got exactly 60 successes, so I do this:

```
prop.test(60,100,0.5,alternative="greater")

##
## 1-sample proportions test with continuity correction
##
## data: 60 out of 100, null probability 0.5
## X-squared = 3.61, df = 1, p-value = 0.02872
## alternative hypothesis: true p is greater than 0.5
## 95 percent confidence interval:
## 0.5127842 1.0000000
## sample estimates:
## p
## 0.6
```

The P-value should at least be fairly small, since 60 is a bit bigger than 50. (Think about tossing a coin 100 times; would 60 heads make you doubt the coin’s fairness? The above says it should.)

- (c) Run `prop.test` again, just as you did before, but this time save the result, and extract the piece of it called `p.value`. Is that the P-value from your test?

Solution: Copying and pasting:

```
p_test=prop.test(60,100,0.5,alternative="greater")
p_test$p.value

## [1] 0.02871656
```

Yep, the same.

- (d) Use `rerun` to estimate the power of a test of $H_0 : p = 0.5$ against $H_a : p > 0.5$ when $n = 500$ and $p = 0.56$, using $\alpha = 0.05$. There are three steps:
- use `rerun` to generate random samples from binomial distributions with $n = 500$ and $p = 0.56$, repeated “many” times (something like 1000 or 10,000 is good)
 - use `map` to run `prop.test` on each of those random samples
 - use `map_dbl` to extract the P-value for each test and save the results (in a vector called, perhaps, `pvals`).

So I lied: the fourth and final step is to count how many of those P-values are 0.05 or less.

Solution: The previous parts, using `rbinom` and `prop.test`, were meant to provide you with the ingredients for this part.

The first step is to use `rbinom`. The first input is 1 since we only want one random binomial each time (the `rerun` will handle the fact that you actually want lots of them). The second step runs `prop.test`; the first input to that is each one of the numbers of successes from the first step. (This is an implied for-each, with each of the simulated binomials playing the role

of “it”, in turn.). The last part is to pull out all the P-values and make a table of them, just like the example in class.

```
rerun(10000,rbinom(1,500,0.56)) %>%
  map(~prop.test(.,500,0.5,alternative="greater")) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>           <int>
## 1 FALSE           1491
## 2 TRUE            8509
```

The estimated power is about 85%. That is, if p is actually 0.56 and we have a sample of size 500, we have a good chance of (correctly) rejecting that $p = 0.5$.

Extra: I know we haven’t learned about SAS yet, but it turns out that SAS can obtain this power by calculation, thus (don’t worry too much about the code yet: that will become clearer later):

```
proc power;
  onesamplefreq
    test=z
    method=normal
    nullproportion=0.5
    proportion=0.56
    sides=U
    ntotal=500
    power=.;
```

The POWER Procedure
Z Test for Binomial Proportion

Fixed Scenario Elements

Method	Normal approximation	
Number of Sides		U
Null Proportion		0.5
Binomial Proportion		0.56
Total Sample Size		500
Variance Estimate	Null Variance	
Alpha		0.05

Computed Power

Power

0.852

SAS says our power is also about 85%, the same thing our simulation said. I took some care to make sure that SAS did the same thing our simulation did (the `test=z`, the `method=normal`, and the `sides=U`). I was actually pleased that my simulation came out so close to the right answer.

In contrast to `power.t.test`, SAS's `proc power` handles power analyses for a lot of things, including analysis of variance, correlation and (multiple) regression. What these have in common is some normal-based theory that allows you (under assumptions of sufficiently normal-shaped populations) to calculate the exact answer (that is, the distribution of the test statistic when the *alternative* hypothesis is true). The case we looked at is one of those because of the normal approximation to the binomial: once n gets big, particularly if p is somewhere near 0.5, the binomial is very well approximated by a normal with the right mean and SD.

The moral of this story is that when you have a decently large sample, $n = 500$ in this case, p doesn't have to get very far away from 0.5 before you can correctly reject 0.5. Bear in mind that sample sizes for estimating proportions need to be larger than those for estimating means, so $n = 500$ is large without being huge. The practical upshot is that if you design a survey and give it to 500 (or more) randomly chosen people, the proportion of people in favour doesn't have to be much above 50% for you to correctly infer that it *is* above 50%, most of the time.

4. We are planning a study to estimate a population mean. The population standard deviation is believed to be 20, and the population distribution is believed to be approximately normal. We will be testing the null hypothesis that the population mean is 100. Suppose the population mean is actually 110, and we want to determine how likely we are to (correctly) reject the null hypothesis in this case, using a two-sided (but one-sample) test with $\alpha = 0.05$.

- (a) We will take a sample of size $n = 30$. Calculate the power of this test.

Solution: `power.t.test`. Fill in: sample size `n`, difference in means `delta` ($10 = 110 - 100$), population SD `sd`, type of test `type` (`one.sample`) and kind of alternative hypothesis `alternative` (`two.sided`). Leave out `power` since that's what we want:

```
power.t.test(n=30,delta=10,sd=20,type="one.sample",alternative="two.sided")

##
##      One-sample t test power calculation
##
##              n = 30
##             delta = 10
##              sd = 20
##      sig.level = 0.05
##              power = 0.7539627
##      alternative = two.sided
```

I meant “calculate” exactly rather than “estimate” (by simulation). Though if you want to, you can do that as well, thus:

```
rerun(1000,rnorm(30,110,20)) %>%
  map(~t.test(.,mu=100)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lg1>             <int>
## 1 FALSE             246
## 2 TRUE              754
```

That came out alarmingly close to the exact answer.

- (b) Find the sample size necessary to obtain a power of at least 0.80 under these conditions. What sample size do you need? Explain briefly how your answer is consistent with (a).

Solution: Again, the implication is “by calculation”.

This time, in `power.t.test`, put in 0.80 for `power` and leave out `n`. The order of things doesn’t matter (since I have named everything that’s going into `power.t.test`):

```
power.t.test(delta=10,power=0.80,sd=20,type="one.sample",alternative="two.sided")

##
##      One-sample t test power calculation
##
##              n = 33.3672
##             delta = 10
##              sd = 20
##      sig.level = 0.05
##              power = 0.8
##      alternative = two.sided
```

To get sample size for power at least 0.80, we have to round 33.36 *up* to the next whole number, ie. $n = 34$ is needed. (A sample of size 33 wouldn’t quite have enough power.)

This answer is consistent with (a) because a sample size of 30 gave a power a bit less than 0.80, and so to increase the power by a little (0.75 to 0.80), we had to increase the sample size by a little (30 to 34).

Estimating sample sizes by simulation is tricky, because the sample size has to be input to the simulation. That means your only strategy is to try different sample sizes until you find one that gives the right power.

In this case, we know that a sample of size 30 doesn't give quite enough power, so we have to up the sample size a bit. How about we try 40? I copied and pasted my code from above and changed 30 to 40:

```
rerun(1000,rnorm(40,110,20)) %>%
  map(~t.test(.,mu=100)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>             <int>
## 1 FALSE             135
## 2 TRUE              865
```

Now the power is a bit too big, so we don't need a sample size quite as big as 40. So probably our next guess would be 35. But before we copy and paste again, we should be thinking about making a function of it first, with the sample size as input. Copy-paste once more and edit:

```
sim_power=function(n) {
  rerun(1000,rnorm(30,110,20)) %>%
    map(~t.test(.,mu=100)) %>%
    map_dbl("p.value") ->
  pvals
  tibble(pvals) %>% count(pvals<=0.05)
}
```

In the grand scheme of things, we might want to have the null and true means, population SD and α be inputs to the function as well, so that we have a more general tool, but this will do for now.

Let's run it with a sample size of 35:

```
sim_power(35)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>             <int>
## 1 FALSE             229
## 2 TRUE              771
```

and I'm going to call that good. (Because there is randomness in the estimation of the power, don't expect to get *too* close to the right answer. This one came out a fair bit less than the right answer; the power for $n = 35$ should be a bit *more* than 0.80.)

Now that you have the software to do it, you can see that figuring out a sample size like this, at least roughly, won't take very long: each one of these simulations takes maybe seconds to run, and all you have to do is copy and paste the previous one, and edit it to contain the new sample size before running it again. You're making the computer work hard while you lazily sip your coffee, but there's no harm in that: programmer's brain cells are more valuable than computer CPU cycles, and you might as well save your brain cells for when you really need

them.

5. This question investigates power by simulation.

- (a) Use `rnorm` to generate 10 random values from a normal distribution with mean 20 and SD 2. Do your values look reasonable? Explain briefly. (You don't need to draw a graph.)

Solution: `rnorm` with the number of values first, then the mean, then the SD:

```
x=rnorm(10,20,2)
```

```
x
```

```
## [1] 21.59476 18.64044 21.83231 18.76556 18.64861 21.81889 21.62614
```

```
## [8] 20.18249 16.91266 20.63490
```

95% of the sampled values should be within 2 SDs of the mean, that is, between 16 and 24 (or 99.7% should be within 3 SDs of the mean, between 14 and 26). None of my values are even outside the interval 16 to 24, though yours may be different.

I saved mine in a variable and then displayed them, which you don't need to do. I did because there's another way of assessing them for reasonableness: turn the sample into z -scores and see whether the values you get look like z -scores (that is, most of them are between -2 and 2 , for example):

```
(x-20)/2
```

```
## [1] 0.79738130 -0.67977910 0.91615386 -0.61722168 -0.67569291
```

```
## [6] 0.90944266 0.81307163 0.09124563 -1.54367207 0.31744905
```

These ones look very much like z -scores. This, if you think about it, is really the flip-side of 68–95–99.7, so it's another way of implementing the same idea.

You might also think of finding the *sample* mean and SD, and demonstrating that they are close to the right answers. Mine are:

```
mean(x)
```

```
## [1] 20.06568
```

```
sd(x)
```

```
## [1] 1.731305
```

The sample SD is more variable than the sample mean, so it can get further away from the population SD than the sample mean does from the population mean.

The downside to this idea is that it doesn't get at assessing the normality, which looking at z -scores or equivalent does. Maybe coupling the above with a boxplot would have helped, had I not said "no graphs", since then you'd (hopefully) see no outliers and a roughly symmetric shape.

This is old-fashioned "base R" technology; you could do it with a data frame like this:

```

d=tibble(x=rnorm(10,20,2))
d
## # A tibble: 10 x 1
##       x
##   <dbl>
## 1  20.6
## 2  19.3
## 3  20.2
## 4  18.7
## 5  21.3
## 6  17.8
## 7  19.8
## 8  20.9
## 9  21.7
## 10 20.6

d %>% summarize(m=mean(x), s=sd(x))
## # A tibble: 1 x 2
##       m      s
##   <dbl> <dbl>
## 1  20.1  1.21

```

These are different random numbers, but are about equally what you'd expect. (These ones are a bit less variable than you'd expect, but with only ten values, don't expect perfection.)

Some discussion about the kind of values you should get, and whether or not you get them, is what is called for here. I want you to say something convincing about how the values you get come from a normal distribution with mean 20 *and* SD 2. "Close to 20" is not the whole answer here, because that doesn't get at "how close to 20?": that is, it talks about the mean but not about the SD.

- (b) Estimate by simulation the power of a *t*-test to reject a null hypothesis of 20 when the true mean is also 20, the population SD is 2, and the sample size is 10, against a (default) two-sided alternative. Remember the steps: (i) generate a lot of random samples from the true distribution, (ii) run the *t*-test with the required null mean, (iii) pull out the P-values, (iv) count how many of them are 0.05 or less.

Solution: Once you get the hang of these, they all look almost the same. This one is easier than some because we don't have to do anything special to get a two-sided alternative hypothesis:

```

rerun(1000,rnorm(10,20,2)) %>%
  map(~t.test(.,mu=20)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)
## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>             <int>
## 1 FALSE             958
## 2 TRUE              42

```

The power is about 4.2%. This seems depressingly small, but see the next part.

- (c) In the simulation you just did, was the null hypothesis true or false? Do you want to reject the null hypothesis or not? Explain briefly why the simulation results you got were (or were not) about what you would expect.

Solution: The null mean and the true mean were both 20: that is, the null hypothesis was correct, and rejecting it would be a mistake, to be precise a type I error. We were doing the test at $\alpha = 0.05$ (by comparing our collection of simulated P-values with 0.05), so we should be making a type I error 5% of the time. This is entirely in line with the 4.2% of (wrong) rejections that I had.

Your estimation is likely to be different from mine, but you should be rejecting about 5% of the time. If your result is very different from 5%, that's an invitation to go back and check your code. On the other hand, if it *is* about 5%, that ought to give you confidence to go on and use the same ideas for the next part.

- (d) By copying, pasting and editing your code from the previous part, estimate the power of the test of $H_0 : \mu = 20$ (against a two-sided alternative) when the true population mean is 22 (rather than 20).

Solution: Here's the code we just used:

```
rerun(1000,rnorm(10,20,2)) %>%
  map(~t.test(.,mu=20)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)
```

One of those 20s needs to become 22. Not the one in the `t.test`, since the hypotheses have not changed. So we need to change the 20 in the `rnorm` line to 22, since that's where we're generating data from the true distribution. The rest of it stays the same:

```
rerun(1000,rnorm(10,22,2)) %>%
  map(~t.test(.,mu=20)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>             <int>
## 1 FALSE             202
## 2 TRUE              798
```

This time, we *want* to reject, since the null hypothesis is false. So look at the **TRUE** count of 798: the power is about $798/1000 \simeq 80\%$. We are very likely to correctly reject a null of 20 when the mean is actually 22.

Another way to reason that the power should be fairly large is to think about what kind of sample you are likely to get from the true distribution: one with a mean around 22 and an SD around 2. Thus the *t*-statistic should be somewhere around this (we have a sample size of 10):

```
t_stat=(22-20)/(2/sqrt(10))
t_stat
## [1] 3.162278
```

and the two-sided P-value should be about

```
2*(1-pt(t_stat,10-1))
## [1] 0.01150799
```

Of course, with your actual data, you will sometimes be less lucky than this (a sample mean nearer 20 or a larger sample SD), but sometimes you will be luckier. But the suggestion is that most of the time, the P-value will be pretty small and you will end up correctly rejecting.

The quantity `t_stat` above, 3.16, is known to some people as an “effect size”, and summarizes how far apart the null and true means are, relative to the amount of variability present (in the sampling distribution of the sample mean). As effect sizes go, this one is pretty large.

- (e) Use R to calculate this power exactly (without simulation). Compare the exact result with your simulation.

Solution: This is `power.t.test`. The quantity `delta` is the difference between true and null means:

```
power.t.test(n=10,delta=22-20,sd=2,type="one.sample",alternative="two.sided")
##
##      One-sample t test power calculation
##
##              n = 10
##             delta = 2
##              sd = 2
##      sig.level = 0.05
##              power = 0.8030962
##      alternative = two.sided
```

This, 0.803, is very close to the 0.798 I got from my simulation. Which makes me think I did them both right. This is not a watertight proof, though: for example, I might have made a mistake and gotten lucky somewhere. But it does at least give me confidence.

Extra: when you estimate power by simulation, what you are doing is rejecting or not with a certain probability (which is the same for all simulations). So the number of times you actually *do* reject has a binomial distribution with n equal to the number of simulated P-values you got (1000 in my case; you could do more) and a p that the simulation is trying to estimate. This is inference for a proportion, exactly what `prop.test` does.

Recall that `prop.test` has as input:

- a number of “successes” (rejections of the null in our case)
- the number of trials (simulated tests)
- the null-hypothesis value of p (optional if you only want a CI)
- (optional) a confidence level `conf.level`.

In part (b), we knew that the probability of (incorrectly) rejecting should have been 0.05 and we rejected 42 times out of 1000:

```
prop.test(42,1000,0.05)

##
## 1-sample proportions test with continuity correction
##
## data: 42 out of 1000, null probability 0.05
## X-squared = 1.1842, df = 1, p-value = 0.2765
## alternative hypothesis: true p is not equal to 0.05
## 95 percent confidence interval:
## 0.03079269 0.05685194
## sample estimates:
## p
## 0.042
```

Looking at the P-value, we definitely fail to reject that the probability of (incorrectly) rejecting is the 0.05 that it should be. Ouch. That's true, but unnecessarily confusing. Look at the confidence interval instead, 0.031 to 0.057. The right answer is 0.05, which is inside that interval, so good.

In part (d), we didn't know what the power was going to be (not until we calculated it with `power.t.test`, anyway), so we go straight for a confidence interval; the default 95% confidence level is fine. We (correctly) rejected 798 times out of 1000:

```
prop.test(798,1000)

##
## 1-sample proportions test with continuity correction
##
## data: 798 out of 1000, null probability 0.5
## X-squared = 354.02, df = 1, p-value < 2.2e-16
## alternative hypothesis: true p is not equal to 0.5
## 95 percent confidence interval:
## 0.7714759 0.8221976
## sample estimates:
## p
## 0.798
```

I left out the 3rd input since we're not doing a test, and ignore the P-value that comes out. (The default null proportion is 0.5, which often makes sense, but not here.)

According to the confidence interval, the estimated power is between 0.771 and 0.822. This interval definitely includes what we now know is the right answer of 0.803.

This might be an accurate enough assessment of the power for you, but if not, you can do more simulations, say 10,000:

```
rerun(10000,rnorm(10,22,2)) %>%
  map(~t.test(.,mu=20)) %>%
  map_dbl("p.value") ->
pvals
tibble(pvals) %>% count(pvals<=0.05)

## # A tibble: 2 x 2
##   `pvals <= 0.05`      n
##   <lgl>             <int>
## 1 FALSE             2004
## 2 TRUE              7996
```

I copied and pasted my code again, which means that I'm dangerously close to turning it into a function, but anyway.

The confidence interval for the power is then

```
prop.test(7996,10000)

##
## 1-sample proportions test with continuity correction
##
## data: 7996 out of 10000, null probability 0.5
## X-squared = 3589.2, df = 1, p-value < 2.2e-16
## alternative hypothesis: true p is not equal to 0.5
## 95 percent confidence interval:
## 0.7915892 0.8073793
## sample estimates:
## p
## 0.7996
```

that is, from 0.792 to 0.807, which once again includes the right answer of 0.803. The first interval, based on 1,000 simulations, has length 0.051, while this interval has length 0.015. The first interval is more than three times as long as the second, which is about what you'd expect since the first one is based on 10 times fewer simulations, and thus ought to be a factor of $\sqrt{10} \simeq 3.16$ times longer.

This means that you can estimate power as accurately as you like by doing a large enough number of simulations. Provided, that is, that you are prepared to wait a possibly long time for it to finish working!

Hand the next two questions in:

6. A student conducts an experiment where she starts a stopwatch and asks a fellow student to tell her when 30 seconds have passed. At that point, she stops the stopwatch and records how much time has actually passed. The student experimenter continues until she has asked thirty different student subjects to guess when 30 seconds are up. The data are in <http://www.utsc.utoronto.ca/~butler/c32/thirty.csv>, with the values separated by commas.
 - (a) (2 marks) Read the data into R, and display the first few lines. You should have two columns, one containing the recorded time for each student, and one containing that student's gender. Is that what you have?
 - (b) (2 marks) Draw a suitable picture that will enable you to compare the centre and spread of the times for the males and females.

- (c) (2 marks) Compare the centres and spreads of guessed times for males and females, using your plot. What do you see? Explain briefly.
 - (d) (3 marks) Carry out an appropriate test to see whether the mean guessed times for males and females are different, given what you observed about spread in the previous part. What do you conclude, in the context of the data?
 - (e) (2 marks) Somebody claims that females are more accurate at guessing when 30 seconds are up than males are. Explain briefly why the data as they currently stand do not help you to address this question, and describe how you might reorganize things to assess this claim. (You do not need to actually assess the claim.)
7. The National Bureau of Standards, some years ago, measured the (mean) selenium content of leaves in one of its orchards to be 0.080 parts per million. This year, a researcher wants to measure selenium content in leaves from the same orchard, to see whether it has changed. The standard deviation of selenium content in a leaf is believed to be 0.005 parts per million, and the distribution of selenium content is believed to be approximately normal.
- (a) (3 marks) Suppose the researcher samples 6 leaves. If the mean selenium content is in fact now 0.075 parts per million, calculate the probability that the researcher will correctly be able to reject a mean of 0.080 parts per million.
 - (b) (2 marks) How many leaves should the researcher sample to bring this power up to 0.90?
 - (c) (3 marks) Repeat part (a), but this time estimating the power using simulation. Is your answer exactly the same as what you got in (a)? Close to it? Comment briefly.

Notes

¹I return to this issue when we look at the same data in SAS later.

²Mine is rather prosaically called `Downloads`.

³Let me know if you have more success than I did.

⁴You might have a second categorical variable by which you want to arrange the facets left and right, and that would go where the dot is.

⁵I took out the `ncol` since that confuses the explanation here.

⁶Google defines this as meaning “outstandingly bad, shocking”.

⁷That would mean assessing whether an observed proportion could be greater than 0.5 just by chance, or whether it is “bigger enough” than 0.5 to reject chance as a plausible explanation.

⁸Two peaks.

⁹Looking back at the data, I see two male observations over 40, with a gap to the next one.

¹⁰This is often the case with the two-sample *t*-test: the group SDs, or the sample sizes, or both, have to be noticeably different for it to matter much.

¹¹The computer scientists among you will note that this is camelCase with the two words “Levene” and “test” smooshed together with the second one capitalized, as opposed to “levene.test” or “levene_test”. It’s called “camel case” because of the uppercase letter T in the middle, which makes it look as if the word has a hump in the middle, like a camel.